DEPLOYGUARD

The future doesn't wait for infrastructure. Neither do we.

Connect your repo. DeployGuard takes it from code to secure, running cloud infrastructure.

Deploy >

DEPLOYGUARD ARCHITECTURE

Authoritative delivery path

LIVE

Repository

GitHub source and commit

Detect & plan

Topology V2 · BuildPlan

Build & publish

GitHub Actions · Docker · ECR

Provision

Terraform · AWS infrastructure

Run & verify

ECS · ALB · exact health evidence

DETECTED APPLICATION PLAN

Frontend | Backend | Managed database when required

Verified live application

Immutable image, runtime identity, routing, and health evidence must agree before promotion.

720 baseline planning states

A deliberately bounded model to show scale - not the real maximum.

4

REPOSITORY TOPOLOGIES

single app / split app / nested / workspace

3

FRONTEND PATTERNS

React-Vite / Next.js / none

5

BACKEND PATTERNS

Express / Django / FastAPI / Flask / none

4

DATABASE MODES

PostgreSQL / MongoDB / external / none

3

CONFIG MODES

build-time / runtime / mixed

4 × 3 × 5 × 4 × 3 = 720

The real space is larger because developers introduce arbitrary folder names, multiple manifests, custom scripts, mixed lockfiles, stale configuration, generated directories and component-specific conventions.

How many stack profiles can one repo represent?

Before folder inconsistency, custom scripts or stale config are even considered.

3 FRONTEND STATES

- React / Vite
- Next.js
- No frontend

5 BACKEND STATES

- Express
- Django
- Flask
- FastAPI
- No backend

4 DATABASE STATES

- PostgreSQL
- MongoDB
- External database
- No database

RAW FRAMEWORK + DB SPACE

3 × 5 × 4 = 60

Subtract 4 database-only states

= 56 meaningful core stack profiles

ADD TOPOLOGY + CONFIG

56 × 4 × 3 = 672

4 repo topologies × 3 config modes

672 bounded planning profiles

Then language-specific package/build/runtime branches expand the space again - so 672 is a bounded baseline, not a maximum.

A framework hit is only one signal.

DeployGuard has to reason about combinations that must run together.

FRONTEND

- React / Vite
- Next.js
- None

BACKEND

- Express
- Django
- FastAPI
- Flask
- None

DATABASE

- PostgreSQL
- MongoDB
- External DB
- None

EXAMPLE VALID PROFILES

→ React/Vite + Express + PostgreSQL

→ Next.js + FastAPI + MongoDB

→ React/Vite + Django + external database

→ Backend-only FastAPI + PostgreSQL

→ Frontend-only React/Vite + no managed database

Real repositories do not obey one template.

The backend has to infer intent from evidence that may be incomplete or contradictory.

WHAT CAN BE INCONSISTENT

- → Misleading folder names
- → Multiple package.json / manifests
- → npm + yarn + pnpm lockfiles
- → Custom build / start scripts
- → Nested apps and workspaces
- → Stale Dockerfiles and old configs
- → Build-time vs runtime env
- → Public / secret / optional values

REAL STACK DETECTION SERVICE

```
You, 2 days ago | 1 author (You)
@Injectable()
export class StackDetectionService {
  private readonly logger = new Logger(StackDetectionService.name);
  private readonly unitDiscovery = new ApplicationUnitDiscoveryService();
  private readonly relationshipInference = new RelationshipInferenceService();

  constructor(
    private readonly templateMatchingService: TemplateMatchingService,
    private readonly deployabilityScanner: RepoDeployabilityScannerService,
    private readonly detectorResolver: MainstreamDetectorResolverService = new MainstreamDetectorResolverService()
  ) {}

  detect(
    workspacePath: string,
    commitSha: string | null,
    preferredAppDirectory?: string | null,
    overrides: ProjectDeploymentOverrides = {},
    includeTopology = true,
  ): DeploymentProfileDraft {
    const scanRoot = this.resolveScanRoot(workspacePath, preferredAppDirectory);
    const preferredDirectory = relative(workspacePath, scanRoot).replace(/\\/g, "/") || ".";
    const manifestFiles = this.findManifestFiles(scanRoot);
    const candidates = this.buildCandidates(scanRoot, manifestFiles);
    const selected = this.selectCandidate(candidates);
    const rankedCandidates = candidates.map((candidate) => ({ directory: candidate.relativeDirectory, score: this.
    const plausibleCandidates = rankedCandidates.filter((candidate) => candidate.score >= 100);
```

candidate discovery → ranking → topology → relationship inference

A wrong assumption can survive for minutes.

Then it looks like a Docker, AWS, routing or health-check problem.

01 **WRONG APP ROOT** → 02 **INCOMPLETE COMPONENT PLAN** → 03 **BUILD MAY STILL PASS** → 04 **INFRASTRUCTURE MAY PROVISION** → 05 **RUNTIME / HEALTH FINALLY FAILS**

WHAT CHANGED

Framework detection → Application understanding under uncertainty

Now the detector combines structure, manifests, dependencies, scripts, runtimes, configuration and component relationships before a deployment plan is trusted.